Spazio Dev S.r.l.
Sviluppo Software ed Elettronica Embedded

Interfaccia UI per dispositivi ESP32 per la visualizzazione di informazioni in tempo reale

Documentazione Tecnica e Specifiche Software

Sviluppatore: Roberto Mariano Doroftei
Versione: 1.0.0
Data: Ottobre 2026
Progetto: Interfaccia UI ed Embedded ESP32
Stato: Approvato
Target Hardware: ESP32

1 Executive Summary

Il presente documento descrive le specifiche tecniche, l’architettura software e il firmware sviluppato per i **dispositivi embedded basati su ESP32** dotati di display, dedicati alla visualizzazione in tempo reale delle informazioni gestite dal sistema CRM aziendale RelAi.

Il sistema consente la fruizione locale di dati chiave (codici QR, prenotazioni, contatori iscritti e partecipanti ad eventi), garantendo al contempo un’elevata affidabilità attraverso meccanismi di provisioning Wi-Fi, caching locale dei dati e resilienza alla disconnessione di rete.

2 Indice

1 Executive Summary 2
2 Indice 3
3 Progettazione e codifica del firmware 1
3.1 Piattaforma e strumenti 1
3.1.1 Partizionamento della flash 1
3.2 Struttura dei moduli 2
3.3 Modello di esecuzione 2
3.3.1 Sincronizzazione LVGL 2
3.3.2 Notifica di aggiornamento dei dati 3
3.4 Display, touch e LVGL (`display_config.cpp`) 3
3.5 Provisioning Wi-Fi e web server locale (`wifi_conessione.cpp`) 4
3.5.1 Modalità di rete 4
3.5.2 Captive portal e file statici 4
3.5.3 Endpoint REST locali 4
3.5.4 Funzioni esposte alla UI e al `main` 5
3.6 Client API del CRM RelAi (`api.cpp`) 5
3.6.1 Configurazione e stato 5
3.6.2 Heartbeat HTTP 5
3.6.3 Canale WebSocket (protocollo Pusher) 6
3.6.4 Comandi remoti e OTA 6
3.6.5 Cache degli eventi 7
3.6.6 Logo dell’organizzazione 7
3.7 Interfaccia grafica (`interfaccia_ui.cpp`) 8
3.7.1 Struttura delle schermate 8
3.7.2 Navigazione 8
3.7.3 Schermata logo 8
3.7.4 Menu e diagnostica 8
3.7.5 Ricostruzione della UI 9
3.8 Flusso dei dati 9
3.9 Persistenza dei dati 9
3.10 Gestione degli errori e resilienza 10
3.11 Considerazioni su sicurezza e limiti noti 10
4 Verifica e validazione 11
4.1 Compilazione e dimensioni 11
4.2 Strategia di verifica 11
4.2.1 Prove funzionali previste 11
4.3 Difetti individuati e risolti 12
4.4 Validazione rispetto ai requisiti 13

3 Progettazione e codifica del firmware

Il presente capitolo documenta l'architettura e l'implementazione del firmware contenuto nella cartella `src/`. Per ogni modulo vengono descritti responsabilità, strutture dati, funzioni pubbliche e comportamento a runtime. I riferimenti a nomi di funzioni, file e costanti corrispondono al codice sorgente.

3.1 Piattaforma e strumenti

Elemento	Valore
Microcontrollore	ESP32-S3 (scheda `esp32-s3-devkitc-1`), 16 MB di flash, PSRAM OPI (`qio_opi`)
Display	AMOLED 466 × 466 px con controller CO5300 su bus QSPI (`Arduino_ESP32QSPI`, libreria *GFX Library for Arduino*)
Touch	Controller capacitivo CST92xx su bus I²C (SDA = GPIO 15, SCL = GPIO 14), gestito da *SensorLib* (`TouchDrvCSTXXX`)
Framework	Arduino per ESP32 su PlatformIO (`platform = espressif32`)
Grafica	LVGL 9.6 con formato colore RGB565, configurazione in `src/lv_conf.h`
Rete	`WiFi`, `WebServer`, `DNSServer`, `HTTPClient`, `WiFiClientSecure`, `WebSocketsClient`, `HTTPUpdate`
Dati	`ArduinoJson` 7, `Preferences` (NVS), `LittleFS`, `LodePNG` (copia locale in `lib/LodePNG`)
Velocità CPU/flash	240 MHz CPU, flash a 80 MHz in modalità QIO, ottimizzazione `-O3`

3.1.1 Partizionamento della flash

La tabella `default_16MB.csv` definisce due slot applicativi per l'aggiornamento OTA e una partizione dati per il file system.

Nome	Tipo	Dimensione	Uso
`nvs`	data/nvs	0x5000 (20 KB)	Namespace `Preferences`: credenziali Wi-Fi, cache eventi
`otadata`	data/ota	0x2000	Selezione dello slot OTA attivo
`app0`, `app1`	app/ota_0, ota_1	0x640000 ciascuna (≈ 6,25 MB)	Due immagini firmware alternate
`spiffs`	data/spiffs	0x360000 (≈ 3,4 MB)	Volume montato come LittleFS: pagine web e `logo.png`
`coredump`	data/coredump	0x10000	Salvataggio del core dump in caso di panic

La dimensione ridotta della partizione NVS (20 KB) è il motivo per cui il logo dell'organizzazione non può essere memorizzato come blob in `Preferences` ma viene salvato come file su LittleFS (si veda la Sezione 3.6.6).

3.2 Struttura dei moduli

File	Responsabilità
`main.cpp`	Punto di ingresso (`setup`), creazione dei due task FreeRTOS, ciclo di gestione rete (`networkTask`) e ciclo di rendering (`displayTask`).
`display_config.cpp/.h`	Inizializzazione del display CO5300, del touch CST92xx e di LVGL; mutex `lvgl_mutex`; luminosità e spegnimento display.
`interfaccia_ui.cpp/.h`	Costruzione delle schermate LVGL (eventi, logo, menu), navigazione, modali, aggiornamento dello stato di sistema.
`api.cpp/.h`	Client del CRM RelAi: heartbeat HTTP, canale WebSocket, cache eventi, comandi remoti, OTA, download e persistenza del logo.
`wifi_conessione.cpp/.h`	Access Point permanente, captive portal, web server di configurazione, endpoint REST locali, reset delle credenziali.
`lv_conf.h`	Configurazione di LVGL (formato colore, memoria, widget e librerie abilitati).

Le dipendenze tra i moduli sono unidirezionali nei punti principali: `main` utilizza tutti gli altri moduli; `wifi_conessione` registra le rotte di `api` tramite `inizializza_api(server)`; `api` notifica la UI con `ui_richiedi_ricarica_tileview()` senza toccare direttamente gli oggetti LVGL.

3.3 Modello di esecuzione

All'avvio `setup()` apre la porta seriale (115200 baud), monta LittleFS e registra le rotte HTTP (`inizializza_sistema_reti()`), inizializza display, touch e LVGL (`init_display_touch_lvgl()`) e infine crea due task FreeRTOS fissati a un core.

Task	Core	Priorità	Stack	Compiti
`NetworkTask`	0	1	24576 B	Avvio AP+STA; `gestisci_wifi_prov()` (web server, DNS, heartbeat, WebSocket); monitoraggio della connessione; download del logo; richiesta di ricostruzione della UI.
`DisplayTask`	1	2	16384 B	Creazione iniziale della UI; ciclo `lv_timer_handler()` con attesa tra 1 e 10 ms.

3.3.1 Sincronizzazione LVGL

LVGL non è thread-safe. Tutte le chiamate alle sue API devono avvenire con il mutex globale `lvgl_mutex` acquisito. Il `displayTask` lo prende per 10 ms a ogni iterazione per eseguire `lv_timer_handler()`; il `networkTask` lo prende, con timeout da 100 ms a 1 s, solo per:

- aggiornare le etichette di stato (`ui_set_connection_status`, `ui_set_wifi_setup_state`), al massimo una volta al secondo;
- mostrare o chiudere il messaggio di errore di rete (`show_wifi_error_msgbox`, `close_wifi_error_msgbox`);
- ricostruire la UI (`create_tileview()`) quando `ui_consuma_richiesta_ricarica()` restituisce `true`.

Regola di progetto: nessuna operazione bloccante di rete o di flash deve essere eseguita mentre il mutex è acquisito, altrimenti il rendering si interrompe. Per questo il download del logo avviene nel `networkTask` fuori dal mutex e la UI legge soltanto dati già presenti in RAM o su LittleFS.

3.3.2 Notifica di aggiornamento dei dati

Il modulo `api` e il `networkTask` non modificano gli oggetti grafici: impostano il flag `ui_reload_requested` (variabile `volatile bool`) chiamando `ui_richiedi_ricarica_tileview()`. Il ciclo del `networkTask` lo consuma con `ui_consuma_richiesta_ricarica()` e, con il mutex acquisito, richiama `create_tileview()`, che distrugge e ricostruisce tutte le schermate leggendo i dati aggiornati.

3.4 Display, touch e LVGL (`display_config.cpp`)

`init_display_touch_lvgl()` esegue in ordine:

1. eventuale alimentazione dell'AMOLED (`AMOLED_PWR_EN`);
2. avvio del display (`gfx->begin(80000000)`), luminosità massima e schermo nero;
3. avvio del touch con `touch.begin(Wire, CST92XX_SLAVE_ADDRESS, IIC_SDA, IIC_SCL)`; `Wire.begin` non viene chiamato esplicitamente perché lo esegue la libreria (una doppia inizializzazione genera l'errore `setPins()` nei log). Se il touch non risponde il firmware entra in un ciclo infinito;
4. `lv_init()` e allocazione di due buffer di rendering da `466 × 233` pixel RGB565 in PSRAM (`heap_caps_aligned_alloc`, allineamento 64 byte, `MALLOC_CAP_SPIRAM`); in caso di allocazione fallita il firmware si blocca;
5. creazione del display LVGL in modalità `LV_DISPLAY_RENDER_MODE_PARTIAL` con doppio buffer e periodo del timer di refresh a 16 ms (≈ 60 FPS);
6. registrazione della callback di flush `my_flush_cb`, che copia l'area aggiornata con `gfx->draw16bitRGBBitmap()`;
7. registrazione di `rounder_cb` sull'evento `LV_EVENT_INVALIDATE_AREA`: le aree di aggiornamento vengono allineate a coordinate di inizio pari e fine dispari, come richiesto dal controller CO5300;
8. creazione del dispositivo di input puntatore (`my_touchpad_read`), che applica le inversioni degli assi `TOUCH_INVERT_X` e `TOUCH_INVERT_Y` (attive) e l'eventuale scambio XY (`TOUCH_SWAP_XY`, disattivo);
9. avvio di un timer hardware (`esp_timer`) che incrementa il tick di LVGL ogni 2 ms;
10. creazione di `lvgl_mutex`.

Le funzioni `imposta_luminosita_display(percentuale)` (0–100, convertita in 0–255) e `spegni_display()` (schermo nero e luminosità 0) acquisiscono a loro volta il mutex con timeout di 1 s. `spegni_display()` è usata dal comando remoto `shutdown`.

3.5 Provisioning Wi-Fi e web server locale (`wifi_conessione.cpp`)

3.5.1 Modalità di rete

`avvia_wifi_provisioning()` configura `WIFI_AP_STA`: l'Access Point resta **sempre attivo** mentre la stazione si connette in background alla rete salvata.

Parametro	Valore
SSID AP	`ESP32-Config`
Password AP	`12345678`
IP AP	`192.168.4.1`, subnet `255.255.255.0`
DNS	`DNSServer` sulla porta 53 che risolve ogni dominio verso l'IP dell'AP (captive portal)
Web server	`WebServer` sulla porta 80
Wi-Fi	`setAutoReconnect(true)`, `setSleep(false)`

Le credenziali della rete di destinazione sono lette dal namespace NVS `wifi-config` (chiavi `ssid` e `password`). Se l'SSID è vuoto il dispositivo resta in sola modalità AP.

3.5.2 Captive portal e file statici

La pagina di configurazione e le risorse (`index.html`, `script.js`, `style.css`, `diagnostica.html`, `diagnostica.js`) risiedono nella cartella `siti_web/`, impostata come `data_dir` di PlatformIO e caricata su LittleFS. `handleFileRead()` serve il file richiesto cercando prima la variante compressa `.gz` (con intestazione `Content-Encoding: gzip`) e provando sia il percorso originale sia quello prefissato da `/siti_web`. `handle_not_found()` risponde con `204` alle sonde di connettività dei sistemi operativi (`generate_204`, `ncsi.txt`, `connecttest.txt`), reindirizza con `302` verso `http://192.168.4.1/` qualsiasi richiesta con `Host` diverso dall'IP dell'AP e, in ultima istanza, risponde `404`.

3.5.3 Endpoint REST locali

Percorso	Metodo	Comportamento
`/`	GET	Serve `index.html` (con redirect captive portal se necessario).
`/scan`	GET	Scansione asincrona delle reti. Risposta `{"scanning": bool, "networks": [ssid...]}`. I duplicati sono rimossi; il parametro `refresh` forza una nuova scansione; una scansione in corso da oltre 15 s viene riavviata. Durante la scansione la STA viene sospesa e poi ripresa (`riprendi_sta_dopo_scan`).
`/status`	GET	Stato di sistema generato da `get_system_status_doc()`: `network_connected`, `ssid`, `firmware`, `elapsed_seconds`, `wifi_configured`, `state_epoch`.
`/connect`	POST	Parametri `ssid` (1–32 caratteri) e `pass` (max 63). Tenta la connessione per al massimo 10 s (interruzione anticipata per `WL_CONNECT_FAILED` dopo 0,5 s e `WL_NO_SSID_AVAIL` dopo 3 s). In

		caso di successo salva le credenziali in NVS e restituisce l'IP; altrimenti restituisce un messaggio d'errore.
`/reset_wifi`	POST	Cancella il namespace `wifi-config` e disconnette la STA (`esegui_reset_wifi`).
`/reboot`	POST	Risponde e riavvia il dispositivo dopo 200 ms.
`/reset_factory`	POST	Ripristino di fabbrica: cancella `wifi-config`, `esp-settings`, `api-cache`, rimuove `/logo.png` e riavvia.
`/api/events`	GET	Registrata da `inizializza_api`: restituisce la cache eventi in formato JSON `{"data": [...]}`.

Il contatore `wifi_state_epoch` viene incrementato a ogni variazione dello stato Wi-Fi (connessione riuscita, reset): le pagine web lo usano per riallinearsi senza dover confrontare lo stato completo.

3.5.4 Funzioni esposte alla UI e al `main`

Funzione	Descrizione
`nome_wifi_salvato()`	Restituisce l'SSID salvato in NVS (stringa vuota se assente).
`wifi_stato_api()`	`1` se la STA è connessa, `0` altrimenti.
`disconnetti_e_cancella_wifi()`	Imposta il flag di reset; l'azione è eseguita da `gestisci_wifi_prov()` nel task di rete.
`avvia_wifi_ibrido()` / `ferma_wifi_ibrido()`	Impostano/azzerano un flag; l'AP è comunque sempre attivo e `wifi_ibrido_attivo()` restituisce `true`.
`gestisci_wifi_prov()`	Eseguita a ogni iterazione del `networkTask`: gestisce il reset, richiama `gestisci_websocket_periodico()`, `dnsServer.processNextRequest()` e `server.handleClient()`.

3.6 Client API del CRM RelAi (`api.cpp`)

3.6.1 Configurazione e stato

Le costanti principali sono definite in testa al file: endpoint di heartbeat (`/api/v1/devices/heartbeat/`), autenticazione dei canali (`/broadcasting/auth`), logo (`/api/organization-logo`), tipo dispositivo (`esp_display`), numero massimo di eventi (`MAX_EVENTS = 10`), intervallo heartbeat di default 30 s, retry dopo un errore 15 s, retry di iscrizione ai canali 10 s. La versione firmware è `FIRMWARE_VERSION` (default `"1.2.0"`, sovrascrivibile da build flag).

Un unico `WiFiClientSecure` è condiviso da heartbeat, autenticazione dei canali e download del logo (`https_client()`); la verifica del certificato è disattivata (`setInsecure()`).

3.6.2 Heartbeat HTTP

`esegui_heartbeat()` invia una `POST` JSON con `Authorization: Bearer <token>` contenente:

```
{ "mac_address": "...", "type": "esp_display",
  "software_version": "1.2.0", "connection_type": "wifi",
  "telemetry": { "uptime_seconds": N, "free_heap_bytes": N, "wifi_rssi": N } }
```

La risposta può contenere:

- `commands`: lista di comandi remoti, eseguiti nell'ordine; se uno è riconosciuto l'elaborazione si interrompe;
- `heartbeat_interval_seconds`: nuovo intervallo, limitato all'intervallo 10–300 s;
- `settings.rotation_interval_seconds`: intervallo di rotazione delle schermate (accettato se ≥ 5 s);
- `settings.events`: elenco eventi, salvato con `salva_eventi()`;
- `websocket`: parametri (`enabled`, `host`, `port`, `app_key`, `channel`, `org_channel`, `scheme`) per avviare il canale in tempo reale, una sola volta.

`gestisci_websocket_periodico()` è chiamata a ogni ciclo del `networkTask`: se il Wi-Fi è connesso fa girare il client WebSocket, tenta le iscrizioni e, alla scadenza di `next_heartbeat_ms` (confronto robusto al wrap-around di `millis()` tramite `scaduto()`) o su richiesta (`richiedi_aggiornamento_api()`), esegue l'heartbeat. In caso di successo il prossimo è pianificato dopo `heartbeat_interval_ms`, in caso di errore dopo 15 s.

3.6.3 Canale WebSocket (protocollo Pusher)

`avvia_websocket()` usa `WebSocketsClient` verso il percorso `/ws/app/<app_key>?protocol=7&client=esp32&version=7.4.0`, con TLS se lo schema è `https`/`wss`, intestazione `Origin` coerente, riconnessione ogni 5 s e heartbeat di livello socket (15 s, timeout 5 s, 2 tentativi).

La gestione dei messaggi (`gestisci_messaggio_ws`) segue il protocollo Pusher:

- `pusher:connection_established`: memorizza `socket_id` e pianifica subito le iscrizioni;
- `pusher:ping`: risponde con `pusher:pong`;
- `pusher_internal:subscription_succeeded`: marca come iscritto il canale del dispositivo o quello dell'organizzazione;
- altri eventi applicativi: il payload (`data`, eventualmente stringa JSON annidata, estratta da `estrai_data`) viene prima valutato come comando remoto e poi come aggiornamento di impostazioni/eventi.

I canali sono due (dispositivo e organizzazione). Per i canali `private-*` e `presence-*` `richiedi_auth_canale()` ottiene la firma tramite `POST` a `/broadcasting/auth` (`socket_id`, `channel_name`); l'iscrizione viene ritentata ogni 10 s finché non è confermata. Alla disconnessione i flag di iscrizione e il `socket_id` vengono azzerati.

3.6.4 Comandi remoti e OTA

`esegui_comando_remoto()` normalizza il nome dell'azione (`estrai_azione` cerca fino a tre livelli di annidamento nei campi `action`, `action_name`, `command_name`, `type`, `name`, `command`; il testo viene reso minuscolo con _ sostituito da `-`) e riconosce quattro comandi:

Azione	Effetto
`restart`	Riavvio dopo 200 ms.
`reset`	Cancellazione di `wifi-config`, `esp-settings`, `api-cache`, rimozione del logo e riavvio.
`shutdown`	Display spento, Wi-Fi disattivato e `esp_deep_sleep_start()`.

`ota`	Aggiornamento firmware dall'URL contenuto in `payload.firmware_url` (o `firmware_url`).

`esegui_aggiornamento_ota()` verifica la connessione, acquisisce `lvgl_mutex` per al più 5 s (per fermare il rendering durante la scrittura), esegue `httpUpdate.update()` con redirect rigorosi e log di avanzamento al 10 %; in caso di successo riavvia il dispositivo.

3.6.5 Cache degli eventi

La struttura `ApiEvent` contiene `id`, `name`, `qr`, `reservations`, `participants`. `salva_eventi()` converte l'array JSON (al massimo 10 elementi), confronta la nuova lista con `events_cache` tramite `eventi_uguali()` e, solo se diversa, aggiorna la RAM, riscrive il namespace NVS `api-cache` (chiavi `ev_id_N`, `ev_name_N`, `ev_qr_N`, `ev_part_N`, `ev_partic_N`; le chiavi degli slot non più usati sono rimosse) e richiede la ricostruzione della UI. Questo evita scritture inutili sulla flash e ricostruzioni superflue.

`api_get_cached_events()` restituisce la cache RAM oppure, dopo un riavvio, la ricostruisce dalla NVS: i dati restano quindi visibili anche senza connessione.

3.6.6 Logo dell'organizzazione

Il logo è un'immagine PNG fornita dal server come PNG binario oppure come testo esadecimale (eventualmente incapsulato in JSON nei campi `logo_hex`, `hex`, `logo`, `image`, `data`). Il flusso è suddiviso per non bloccare la UI.

Funzione	Comportamento
`api_logo_aggiorna()`	Bloccante, da chiamare fuori dal mutex LVGL. Richiede il logo (timeout connessione 3 s, risposta 5 s, dimensione massima 1 MB), riconosce un PNG binario dalla firma o decodifica l'esadecimale (`decodifica_hex_png`, che ammette separatori e prefissi `0x`) e valida firma e dimensioni (larghezza e altezza da 1 a 1024). Se il contenuto è diverso da quello in cache lo salva in `/logo.png` su LittleFS e restituisce `true`.
`api_get_organization_logo_bytes()`	Non bloccante. Restituisce la copia in RAM; se vuota la carica da `/logo.png` verificando la firma PNG.
`api_logo_scaricato()`	`true` dopo almeno un download riuscito.
`api_logo_elimina()`	Rimuove `/logo.png` e azzera cache e flag; è chiamata dai reset di fabbrica (web, comando remoto, pulsante sul display).

Nel `networkTask`, finché `api_logo_scaricato()` è `false` e il Wi-Fi è connesso, viene eseguito un solo tentativo ogni 30 s; se il logo cambia viene richiesta la ricostruzione della UI. Senza connessione, o al riavvio, il logo già scaricato continua a essere mostrato perché persistente su LittleFS.

Motivazione della scelta (problema risolto). In una versione precedente il logo (≈ 30 KB) veniva scritto con `Preferences::putBytes()` dentro `create_logo_ui()`. La partizione NVS è di soli 20 KB: la scrittura falliva e il messaggio di errore della libreria causava un `LoadProhibited` nella `log_printf`, con riavvio continuo. Inoltre la richiesta HTTP era eseguita con il mutex

LVGL acquisito, bloccando il rendering. La soluzione adottata è la persistenza su LittleFS e il download nel task di rete.

3.7 Interfaccia grafica (`interfaccia_ui.cpp`)

3.7.1 Struttura delle schermate

La UI è composta da schermate LVGL indipendenti su sfondo nero, caricate con animazione. Gli indici sono: **eventi** (da 0 a N − 1), **logo** (indice N) e **menu** (indice N + 1), con `total_screens_count = N + 2`. Se la cache è vuota viene creato un evento segnaposto «In attesa di eventi Cloud…».

Ogni pagina evento (`create_prototype_event_ui`) usa un `lv_tileview` con due tile affiancate: la prima mostra ID, nome, prenotazioni e partecipanti; la seconda un codice QR (`lv_qrcode`, 200 px) generato dal campo `qr` dell'evento, oppure «QR non disponibile». Le informazioni di ogni pagina sono salvate in `PageDataUI`; `get_num_dynamic_pages()` e `get_event_id_for_page()` espongono il numero di pagine e l'ID evento.

3.7.2 Navigazione

- **Barra laterale fissa** (`create_global_fixed_sidebar`) sul layer superiore con due pulsanti circolari (su/giù) che calcolano l'indice precedente o successivo in modo ciclico e caricano la schermata con animazione `OVER_BOTTOM` / `OVER_TOP`.
- **Rotazione automatica** (`auto_rotate_timer_cb`): un `lv_timer` con periodo pari a `rotation_interval_seconds` del CRM avanza alla schermata successiva.
- **Protezione dalle transizioni sovrapposte**: il cambio schermata viene differito con un timer a 1 ms (`do_screen_change_cb`), viene azzerato lo stato dell'indev (`lv_indev_reset`) e il flag `is_animating` è liberato dopo 350 ms (`unlock_animation_cb`); durante l'animazione i comandi di navigazione sono ignorati.
- **Triplo tocco**: tre tocchi entro 1,5 s su qualsiasi schermata (`user_interaction_cb`) attivano o disattivano la rotazione automatica. L'alone dei pulsanti (`update_nav_buttons_glow`) è verde (`0x57E389`) a rotazione attiva e rosso (`0xD60429`) quando è sospesa o è aperta una finestra modale.

3.7.3 Schermata logo

`create_logo_ui()` legge i byte del PNG da `/logo.png` su LittleFS, li decodifica con `lodepng::decode()` in RGBA, li converte in RGB565 in un buffer `std::vector<uint16_t>` e compila un `lv_img_dsc_t` (formato `LV_COLOR_FORMAT_RGB565`, `stride = larghezza × 2`) mostrato in un `lv_image` da 225 × 300 px con allineamento `CONTAIN`. La decodifica avviene una sola volta (`logo_dsc_inizializzato`); senza logo compare «LOGO NON DISPONIBILE». Il decoder PNG integrato di LVGL è disattivato (`LV_USE_LODEPNG 0`) per evitare duplicazioni di simboli con la libreria esterna.

3.7.4 Menu e diagnostica

La schermata menu contiene sei pulsanti:

Pulsante	Azione

Stato Sistema	Apre una finestra con rete, stato, ultima sincronizzazione e versione firmware, aggiornate dal `networkTask` una volta al secondo tramite `ui_set_connection_status()`.
QR Setting ESP32	Mostra il QR `WIFI:S:ESP32-Config;T:WPA;P:12345678;;` per collegarsi all'AP di configurazione.
QR Condivisione Rete	Mostra il QR con SSID e password della rete a cui il dispositivo è connesso (`WIFI:S:...;T:WPA;P:...;;`); se disconnesso un QR «Disconnesso».
Reset Wi-Fi	Richiama `disconnetti_e_cancella_wifi()`.
Riavvio ESP32	Riavvio differito di 300 ms tramite timer LVGL.
Reset ESP32	Cancella `wifi-config` e `api-cache`, elimina il logo e riavvia (timer da 300 ms).

Le finestre modali (`lv_msgbox` sul layer superiore) sospendono la rotazione automatica e il relativo alone, che vengono ripristinati alla chiusura. Se la STA resta disconnessa da oltre 15 s con una rete salvata, il `networkTask` mostra il messaggio «Avviso di Rete», chiuso automaticamente al ripristino della connessione.

3.7.5 Ricostruzione della UI

`create_tileview()` crea una schermata temporanea, elimina modali, schermate, barra laterale e pulsanti precedenti, rilegge gli eventi dalla cache, ricrea eventi, logo, menu e barra laterale, carica la prima pagina, elimina la schermata temporanea e ricrea il timer di rotazione. Viene invocata sempre con `lvgl_mutex` acquisito, sia al primo avvio sia a ogni aggiornamento.

3.8 Flusso dei dati

```
CRM RelAi --(HTTPS heartbeat / WSS Pusher)--> api.cpp
    api.cpp: salva_eventi() --> NVS "api-cache"  +  flag ricarica UI
    api.cpp: api_logo_aggiorna() --> LittleFS "/logo.png"  +  flag ricarica UI
networkTask: consuma flag --> [lvgl_mutex] create_tileview()
create_tileview(): api_get_cached_events() + lettura /logo.png
displayTask: [lvgl_mutex] lv_timer_handler() --> flush --> display
```

3.9 Persistenza dei dati

Supporto	Chiave/per-corso	Contenuto
NVS `wifi-config`	`ssid`, `password`	Credenziali della rete di destinazione.
NVS `api-cache`	`ev_*_N`	Ultimi eventi ricevuti (max 10).
NVS `esp-settings`	—	Impostazioni; cancellato dal ripristino di fabbrica.
LittleFS	`/logo.png`	Logo dell'organizzazione.
LittleFS	`/siti_web/*`	Pagine del captive portal.

3.10 Gestione degli errori e resilienza

- **Disconnessione Wi-Fi:** riconnessione automatica del driver, ritentata dal firmware ogni 5 s rileggendo l'SSID salvato, avviso a display dopo 15 s, dati mostrati dalla cache.
- **Errori HTTP/WebSocket:** heartbeat ritentato dopo 15 s, iscrizioni ai canali ogni 10 s, riconnessione WebSocket ogni 5 s con ping periodico.
- **Input non valido:** `/connect` valida la lunghezza di SSID e password; le risposte JSON dei servizi sono controllate prima dell'uso (`deserializeJson`); il logo è accettato solo con firma PNG e dimensioni plausibili.
- **Scritture su flash:** la cache eventi e il logo vengono riscritti solo se il contenuto cambia.
- **Risorse:** ogni secondo il `networkTask` stampa su seriale memoria libera, blocco più grande e frammentazione (`log_heap_status`).

3.11 Considerazioni su sicurezza e limiti noti

I punti seguenti derivano dall'analisi del codice e vanno considerati nelle evoluzioni del prototipo:

- il token API del CRM, l'indirizzo MAC del dispositivo e gli URL sono definiti come costanti nel sorgente (`api.cpp`); in produzione andrebbero provisionati per dispositivo e memorizzati in modo protetto;
- la verifica del certificato TLS è disattivata (`setInsecure()`): la comunicazione è cifrata ma il server non è autenticato;
- l'Access Point usa SSID e password fissi, riportati nel QR mostrato dal menu; gli endpoint locali di reset, riavvio e connessione non richiedono autenticazione per chi è collegato all'AP;
- la password Wi-Fi è salvata in chiaro in NVS e inclusa nel QR di condivisione;
- il flusso di provisioning `/connect` è bloccante fino a 10 s nel contesto del web server;
- se touch o PSRAM non sono disponibili il firmware entra intenzionalmente in un ciclo infinito (nessun riavvio automatico);
- `gestisci_aggiornamento_api_periodico()` e `ui_load_cached_data()` sono funzioni vuote mantenute per compatibilità.

4 Verifica e validazione

4.1 Compilazione e dimensioni

Il firmware viene compilato con `platformio run` (ambiente `esp32-s3-devkitc-1`). L'ultima compilazione completata senza errori ha prodotto l'immagine unica `firmware.factory.bin` (bootloader, tabella delle partizioni, `boot_app0` e applicazione). L'occupazione di memoria riportata da PlatformIO è:

Risorsa	Utilizzo	Percentuale
RAM	52 184 byte su 327 680 (dati statici)	15,9 %
Flash	1 778 123 byte su 6 553 600 (slot applicativo)	27,1 %

Lo spazio libero nello slot applicativo (oltre il 70 %) rende ampio il margine per gli aggiornamenti OTA. I buffer di rendering (due da 466 × 233 pixel RGB565) e i dati dinamici risiedono in PSRAM/heap e non sono inclusi nella percentuale di RAM statica; a runtime il `networkTask` stampa ogni secondo `[HEAP] Free / Largest block / Frag`, utile per verificare l'assenza di perdite e di frammentazione.

4.2 Strategia di verifica

La cartella `test/` del progetto PlatformIO contiene soltanto il file `README` predefinito: non sono presenti test automatici. La verifica è svolta con prove funzionali manuali sul dispositivo, osservando il monitor seriale (115200 baud, con decodifica delle eccezioni `esp32_exception_decoder`) e il comportamento a display. I log del firmware usano prefissi per modulo (`[Wi-Fi]`, `[API]`, `[WebSocket]`, `[Cache]`, `[OTA]`, `[LittleFS]`, `[HEAP]`, `[UI]`) per facilitare l'analisi.

4.2.1 Prove funzionali previste

ID	Area	Procedura e risultato atteso
T1	Avvio	Dopo il reset compaiono `[LittleFS] ... montato` e `[Wi-Fi] ... routing HTTP configurato`; l'AP `ESP32-Config` è visibile.
T2	Provisioning	Collegarsi all'AP, aprire `http://192.168.4.1/`, selezionare una rete e inserire la password: il dispositivo si connette e le credenziali restano dopo il riavvio.
T3	Credenziali errate	Inserire una password errata: `/connect` risponde con un messaggio d'errore entro 10 s e l'AP rimane attivo.
T4	Eventi	Dopo la connessione compare `[Cache] Heartbeat: N eventi aggiornati` e le pagine evento mostrano ID, nome, prenotazioni, partecipanti e QR.
T5	Tempo reale	Modificare un evento sul CRM: la UI si aggiorna dopo il messaggio WebSocket senza riavvio.

T6	Persistenza offline	Riavviare il dispositivo con il router spento: eventi (NVS) e logo (LittleFS) vengono mostrati dalla cache.
T7	Logo	Con rete disponibile compare `[API] Logo PNG caricato`; la schermata logo mostra l'immagine e la navigazione resta fluida durante il download.
T8	Perdita rete	Spegnere il router: dopo 15 s compare «Avviso di Rete», che scompare al ripristino.
T9	Navigazione	Pulsanti su/giù, rotazione automatica e triplo tocco (verde = rotazione attiva, rosso = sospesa).
T10	Comandi remoti	`restart`, `reset`, `shutdown` e `ota` inviati dal CRM producono l'effetto descritto nel Sezione 3.6.4.
T11	Ripristino di fabbrica	Da web (`/reset_factory`), da comando remoto o dal pulsante «Reset ESP32»: dopo il riavvio non restano credenziali, eventi né `/logo.png`.

I risultati di ogni prova vanno registrati in fase di collaudo; il presente documento non riporta esiti numerici che non siano stati misurati.

4.3 Difetti individuati e risolti

ID	Sintomo e causa	Correzione
D1	Riavvio continuo con `Guru Meditation Error: LoadProhibited` nel backtrace `Preferences::putBytes ← create_logo_ui ← create_tileview`. Causa: scrittura del logo (≈ 30 KB) nella partizione NVS da 20 KB.	Logo salvato come file `/logo.png` su LittleFS; NVS non più usata per il logo.
D2	Interfaccia che «laggava» alla creazione della schermata logo. Causa: richiesta HTTP e decodifica esadecimale eseguite con `lvgl_mutex` acquisito.	Download spostato nel `networkTask` fuori dal mutex; la UI legge solo il file già salvato.
D3	Primo tentativo di download con `HTTP -1` all'avvio. Causa: richiesta eseguita prima che la rete fosse pronta, con attese bloccanti fino a 10 s tra i tentativi.	Download tentato solo a Wi-Fi connesso, un tentativo ogni 30 s, senza attese bloccanti.
D4	Il logo scaricato non compariva a display. Causa: percorso `L:/logo.png` senza driver filesystem LVGL registrato; in seguito, build fallita per i file di test/benchmark della libreria LodePNG (richiesta di `SDL/SDL.h`).	Decodifica PNG con LodePNG e conversione in RGB565 in memoria; libreria ridotta a `lodepng.cpp/.h` in `lib/LodePNG`.
D5	Il ripristino di fabbrica non eliminava il logo.	`api_logo_elimina()` chiamata nei tre percorsi di reset (web, comando remoto, pulsante sul display).

4.4 Validazione rispetto ai requisiti

La tracciabilità tra requisiti e casi d’uso è riportata nel capitolo di analisi dei requisiti. Le prove T2 – T11 coprono i requisiti obbligatori di provisioning, visualizzazione degli eventi, funzionamento in assenza di rete, diagnostica locale e comandi remoti; il collaudo finale sul dispositivo conferma o smentisce ciascun requisito.